Exercícios aula 4

Lucca Giordano – 23024522

Exercício 1:

Primeiro, a implementação do algoritmo de Dijkstra:

```
def dijkstra(grafo, origem):
    n = len(grafo)
    dist = [float('inf')] * n
    dist[origem] = 0

    heap = []
    heapq.heappush(heap, (0, origem))

    while heap:
        dist_atual, u = heapq.heappop(heap)

        if dist_atual > dist[u]:
            continue

        for v, peso in grafo[u]:
            nova_dist = dist[u] + peso
            if nova_dist < dist[v]:
                dist[v] = nova_dist
                heapq.heappush(heap, (nova_dist, v))

    return dist
```

Segundo, a criação das entradas:

```
N = 8  # Número de vértices
M = 10 # Número de arestas

# Arestas (u, v, peso):
arestas = [
    (1, 2, 4),
    (1, 3, 1),
    (2, 3, 2),
    (2, 4, 5),
    (3, 4, 8),
    (3, 5, 10),
    (4, 5, 2),
    (4, 6, 6),
    (5, 6, 3),
```

```
    (7, 8, 1)
]

Q = 5 # N de consultas

# Consultas (origem, destino):
consultas = [
    (1, 6),
    (1, 5),
    (2, 6),
    (1, 8),
    (7, 8)
]
```

Depois, para as consultas:

```
for s, t in consultas:
    if s not in cache_distancias:
        cache_distancias[s] = dijkstra(grafo, s)

    custo_minimo = cache_distancias[s][t]

    if custo_minimo == float('inf'):
        print(f"De {s} para {t}: IMPOSSIVEL")
    else:
        print(f"De {s} para {t}: {custo_minimo}")
```

E por último, o resultado do algoritmo:

De 1 para 6: 13

De 1 para 5: 10

De 2 para 6: 10

De 1 para 8: IMPOSSIVEL

De 7 para 8: 1

Exercício 2:

Para obter a Árvore Geradora Mínima de um grafo, o Kruskal é um bom algoritmo. Sua implementação se dá por:

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
```

```
        self.rank = [0] * n

    def find(self, a):
        while self.parent[a] != a:
            self.parent[a] = self.parent[self.parent[a]]
            a = self.parent[a]
        return a

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)

        if ra == rb:
            return False

        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra

        self.parent[rb] = ra

        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1

        return True

def kruskal_mst(n, edges):
    uf = UnionFind(n + 1)
    edges_sorted = sorted(edges, key=lambda x: x[2])

    mst_edges = []
    total = 0

    for u, v, w in edges_sorted:
        if uf.union(u, v):
            mst_edges.append((u, v, w))
            total += w
            if len(mst_edges) == n - 1:
                break

    if len(mst_edges) != n - 1:
        raise ValueError("O grafo não é conexo — MST não existe
(há floresta).")

    return total, mst_edges
```

Segundo, a criação das entradas:

```
N = 6
```

```
    M = 9

    arestas = [
        (1, 2, 4),
        (1, 3, 2),
        (2, 3, 1),
        (2, 4, 5),
        (3, 4, 8),
        (3, 5, 10),
        (4, 5, 2),
        (4, 6, 6),
        (5, 6, 3)
    ]
```

E por último, o cálculo da MST:

```
try:
    custo_total, _ = kruskal_mst(N, arestas)
    print(custo_total)
except ValueError:
    print("IMPOSSIVEL")
```

O algoritmo resulta em:

13

Exercício 3:

Uma alternativa ao Kruskal é o Prim, que também calcula a MST de um grafo. Sua implementação é dada por:

```
def prim_mst(adj, start=1):
    n = len(adj) - 1
    visited = [False] * (n + 1)
    mst_edges = []
    total = 0

    heap = [(0, -1, start)]

    while heap and len(mst_edges) < n - 1:
        w, u, v = heapq.heappop(heap)

        if visited[v]:
            continue
```

```
        visited[v] = True
        total += w

        if u != -1:
            mst_edges.append((u, v, w))

        for to, wt in adj[v]:
            if not visited[to]:
                heapq.heappush(heap, (wt, v, to))

    if not all(visited[1:]):
        raise ValueError("O grafo não é conexo — MST não existe
(há floresta).")

    return total, mst_edges
```

As entradas são as mesmas usadas no exercício anterior, a verificação é dada por:

```
    try:
        custo_total, _ = prim_mst(adj, start=1)
        print(f"{custo_total}")
    except ValueError:
        print("IMPOSSIVEL")
```

E isso resulta em:

13